

Programação Orientada a Objetos
Trabalho Prático 1:
Cromos Fifa 2022

Relatório

Martim Lourenço | nº22005229

José Terremoto | nº 22109272

Docente:

Prof. Doutor Tiago Candeias

2022

Conteúdo

Introdução.....	3
Abordagem.....	3
Diagrama de Classes (UML).....	4
Análise do diagrama.....	5
Testes	7
Classe ClientTest.....	7
Conclusão	10

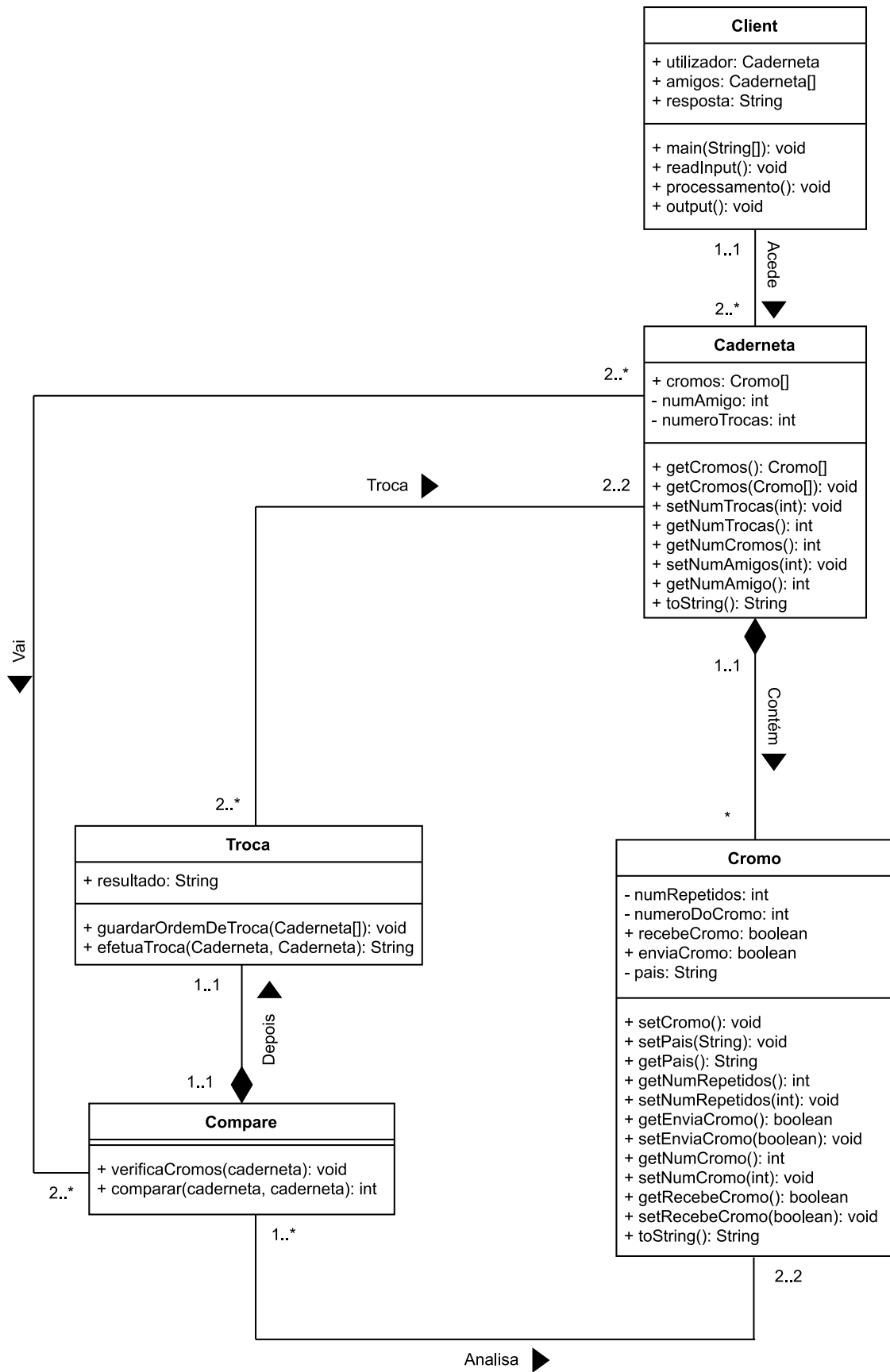
Introdução

Este trabalho foi idealizado para a disciplina de Programação Orientada a Objetos. Ele consiste num programa feito em Java que representa cadernetas onde o utilizador insere um ficheiro .txt com cromos do utilizador e dos seus amigos, sendo que os cromos são identificados por 3 letras que representam o país e o número do cromo. Ao executar o programa, ele realizará trocas de cromos entre as cadernetas do utilizador e a dos amigos, por ordem de quem tem mais cromos para troca, ou seja, a caderneta com mais cromos para troca será a primeira a trocar. O programa termina após todas as trocas possíveis terem sido realizadas, mostrando na consola todos os amigos com quem o utilizador trocou, os cromos que foram trocados e os cromos em falta na coleção do utilizador.

Abordagem

A nossa abordagem começou após a leitura do enunciado, identificando as classes iniciais: **Caderneta**, **Cromo** e **Cliente**. De seguida fizemos um diagrama UML muito simplificado que teve de ser reestruturado mais tarde, porque ao longo do desenvolvimento do programa acabamos por criar mais 2 classes: **Troca** e **Compare**. Essas classes foram concebidas porque queríamos criar um projeto que fosse o mais modular possível. Fizemos testes na classe Cliente para analisar todos os nossos métodos e após a confirmação de tudo funcionar bem, documentamos usando o Javadoc, formatamos o código e por fim, escrevemos este relatório.

Diagrama de Classes (UML)



Análise do diagrama

1. **Client:** É nesta classe onde se encontra o **main()** e mais três métodos: o **readInput()** que recebe o input, que neste caso é um ficheiro com o número de cromos da seleção, os cromos do utilizador, a quantidade de amigos e os cromos dos amigos. O **processamento()** que envia as informações anteriormente referidas para as outras classes para serem processadas. E por último o **output()** que faz com que sejam mostradas as trocas que ocorreram entre o utilizador e os amigos na consola do IDE.
2. **Caderneta:** Como o nome indica, esta classe representa uma caderneta de cromos, quer seja do utilizador ou de um amigo. Esta classe possui o construtor **Caderneta()** e getters e setters que vão devolver ou modificar os atributos do construtor da caderneta, ou seja, para haver encapsulamento.
3. **Cromo:** Tal como foi explicado anteriormente, esta classe é uma representação, que neste caso é de um cromo, que possui um construtor, getters e setters, mas desta vez ao nível do cromo.
4. **Compare:** O **Compare** possui apenas 2 métodos: **verificaCromos()** e o **comparar()**. O **verificaCromos()** percorre os cromos de uma das cadernetas e utilizando o getter **getNumRepetidos** para cada cromo, se for menor que 1, vai colocar numa string o país do cromo e o número desse cromo, usando os getters **getPais** e **getNumCromo**. Enquanto o **comparar()** vai comparar 2 cadernetas: a caderneta **A** (utilizador) e a caderneta **B** (amigo), percorrendo todos os cromos. Se o utilizador não tiver um certo cromo e o amigo tiver mais do que 1 desse cromo, o booleano do **recebeCromo** do **A** como True e o **enviarCromo** da caderneta do **B** vai ficar True, incrementando o número de cromos que o utilizador tem de receber. Também acontece ao contrário: Caso seja a caderneta **B** que não possua um cromo e o utilizador possua mais do que 1 desse cromo, então o booleano do **recebeCromo** do **B** fica True, enquanto o booleano do **enviarCromo** do **A** fica True, incrementando o número de cromos que o utilizador tem de enviar. Como o número de cromos a receber

tem de ser igual ao número de cromos a enviar para haver troca de cromos, caso que um dos números sejam menores, ele vai retornar o número menor e vai colocar-lho no número de trocas da caderneta **B**, usando o setter **setNumTrocas()**.

5. **Troca:** A classe **Troca** possui 2 métodos: o **guardarOrdemDeTroca()**, que guarda o numero do amigo na devida caderneta, usando o setter **setNumAmigo**, sendo que depois reordena as cadernetas de maneira que o amigo com a maior quantidade de cromos para trocar será o primeiro a trocar, ou seja, se o número de trocas de um amigo 3 for maior que a do amigo 2, copia a caderneta menor para um caderneta temporária, copia a caderneta maior para a caderneta menor, e, por fim, copia a caderneta temporária para a caderneta maior, trocando as posições de array das cadernetas. O **efetuaTroca()** percorre os cromos do amigo e utilizando o **setRecebeCromo** e o **setEnviaCromo**, vai colocar estes como False. Depois vai voltar a fazer o **comparar()**, porque ao fazer a troca com um amigo, pode não conseguir fazer as mesmas trocas com o amigo a seguir porque já realizou anteriormente percorrendo os cromos das cadernetas. Se o utilizador tiver o atributo **enviaCromo** como True e o amigo tiver o atributo **recebeCromo** como True. Se o número de cromos enviado pelo utilizador for menor que o número de trocas a efetuar com esse amigo (para não dar mais cromos de que é suposto), se o número de repetidos do utilizador for maior que um e o número de repetidos do amigo for igual a 0, então vai subtrair 1 ao número de cromos repetidos do utilizador, aumentar o número de cromos repetidos de um cromos específico do amigo e vai incrementar o número de cromos enviado pelo utilizador. Também vai colocar o país e o número desse cromos numa array de string **darCromo** no índice p e incrementar p. Isto também acontece ao contrário, ou seja, há uma subtração no número de cromos repetidos do amigo e adiciona-se ao cromos do utilizador. Por fim, vai-se copiar os conteúdos do array de Strings **obterCromo** e **darCromo** para a String resultado, retornando essa String.

Testes

Classe ClientTest

- `testReadInput1()`: Testa o input do ficheiro `input1.txt`, analisa se foram lidos cromos dos 5 amigos e verifica que o número de cromos dos amigos e do utilizador são iguais.
- `testReadInput2()`: Testa o input do ficheiro `input2.txt`, analisa se foram lidos cromos dos 5 amigos e verifica que o número de cromos dos amigos e do utilizador são iguais.
- `testReadInput3()`: Testa o input do ficheiro `input3.txt`, analisa se foram lidos cromos dos 5 amigos e verifica que o número de cromos dos amigos e do utilizador são iguais.
- `testProcessamento1()`: Usando o `input1.txt`, vai ver se o número de trocas efetuadas com cada amigo é igual ao número de trocas efetuadas com cada amigo dados pelo output.
- `testProcessamento2()`: Usando o `input2.txt`, vai ver se o número de trocas efetuadas com cada amigo é igual ao número de trocas efetuadas com cada amigo dados pelo output.
- `testProcessamento3()`: Usando o `input3.txt`, vai ver se o número de trocas efetuadas com cada amigo é igual ao número de trocas efetuadas com cada amigo dados pelo output.

- `testReadOutput1()`: Testado para o `input1.txt`, foi copiado o `output1.txt` para uma string e verifica-se se é igual à resposta que aqui é imprimida e que é obtida pelo processamento.
- `testReadOutput2()`: Testado para o `input2.txt`, foi copiado o `output2.txt` para uma string e verifica-se se é igual à resposta que aqui é imprimida e que é obtida pelo processamento.
- `testReadOutput3()`: Testado para o `input3.txt`, foi copiado o `output3.txt` para uma string e verifica-se se é igual à resposta que aqui é imprimida e que é obtida pelo processamento.

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
ano 2_Cromos	95.7 %	1,160	52	1,212
ano 2_Cromos	95.7 %	1,160	52	1,212
(default package)	95.7 %	1,160	52	1,212
Caderneta.java	100.0 %	67	0	67
Client.java	88.8 %	183	23	206
ClientTest.java	97.9 %	423	9	432
Compare.java	97.7 %	128	3	131
Cromo.java	75.0 %	42	14	56
Troca.java	99.1 %	317	3	320

Figura A – O coverage do projeto.

Finished after 0.13 seconds	
Runs: 9/9	Errors: 0
Failures: 0	
ClientTest [Runner: JUnit 5] (0.073 s)	Failure Trace
testReadInput1() (0.029 s)	
testReadInput2() (0.010 s)	
testReadInput3() (0.007 s)	
testOutput1() (0.003 s)	
testOutput2() (0.006 s)	
testOutput3() (0.005 s)	
testProcessamento1() (0.001 s)	
testProcessamento2() (0.003 s)	
testProcessamento3() (0.004 s)	

Figura B – Os testes da classe ClientTest.

ClientTest [Runner: JUnit 5] (0.084 s)
testReadInput1() (0.028 s)
testReadInput2() (0.013 s)
testReadInput3() (0.010 s)
testOutput1() (0.004 s)
testOutput2() (0.005 s)
testOutput3() (0.011 s)
testProcessamento1() (0.001 s)

Figura C – Imagem dos testes da classe ClientTest com mais zoom.

Conclusão

Com este trabalho pudemos colocar em prática alguns conceitos que aprendemos nesta disciplina, testando os nossos conhecimentos programação em Java.

Tivemos algumas dificuldades ao longo deste projeto, principalmente nos construtores, encapsulamento, nas exceções e nos testes. Podíamos ter feito mais testes, mais exceções e talvez simplificado e otimizado o código. Mas mesmo assim, conseguimos ultrapassar uma grande parte desses obstáculos, melhorando o nosso trabalho em equipa no processo, adquirindo mais conhecimento e competências para o futuro.